

Tree

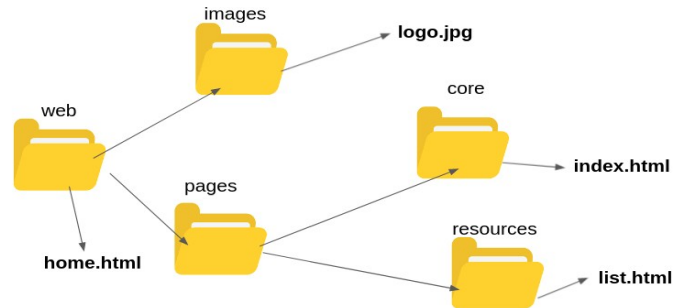
A tree is a non-linear hierarchical data structure which is a **connected acyclic graph**.



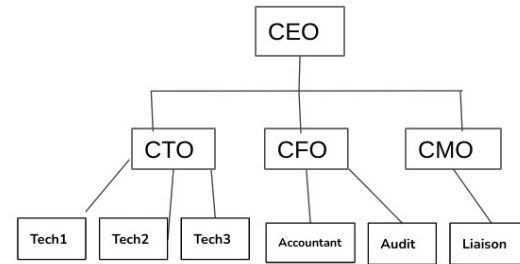
Tree Introduction(next)

Trees allow us to implement a host of algorithms much faster than when using linear data structures, such as lists, vectors, and sequences. Trees also provide a natural organization for data, and consequently have become ubiquitous structures in file systems, graphical user interfaces, databases, Web sites, and other computer systems. It is not always clear what productivity experts mean by “nonlinear” thinking, but when we say that trees are “nonlinear,” we are referring to an organizational relationship that is richer than the simple “before” and “after” relationships between objects in sequences.

Scenario 1: Suppose you want to store a file system. You have the root directory and the root has sub directories and sub directories can store other directories or store files in the following form.



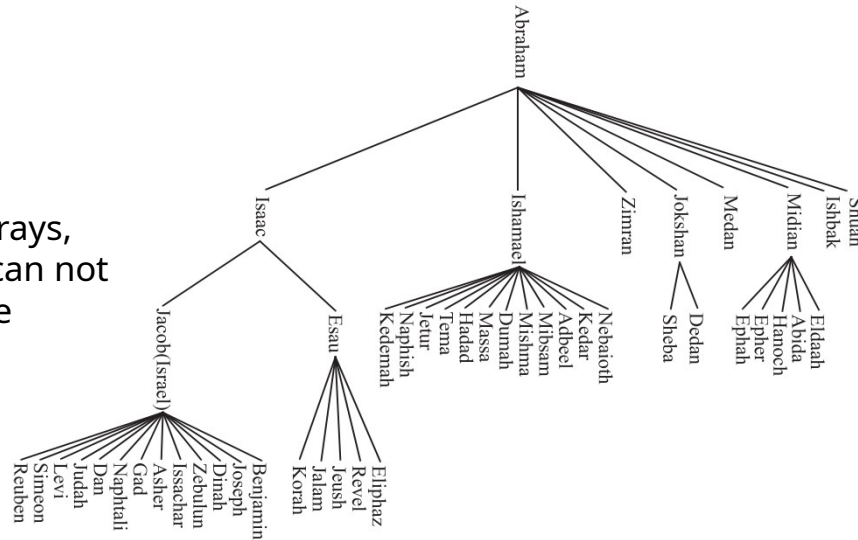
Scenario 2: Suppose you want to store a company hierarchy. You have a CEO-Chief Executive Officer and three people reporting to CEO which are CTO-Chief Technical Officer, CFO-Chief Finance Officer and CMO-Chief Marketing Officer. Under the CTO there are other employees reporting to them.



Linear data structures like arrays, linked list, Queue and Stack can not store these data.

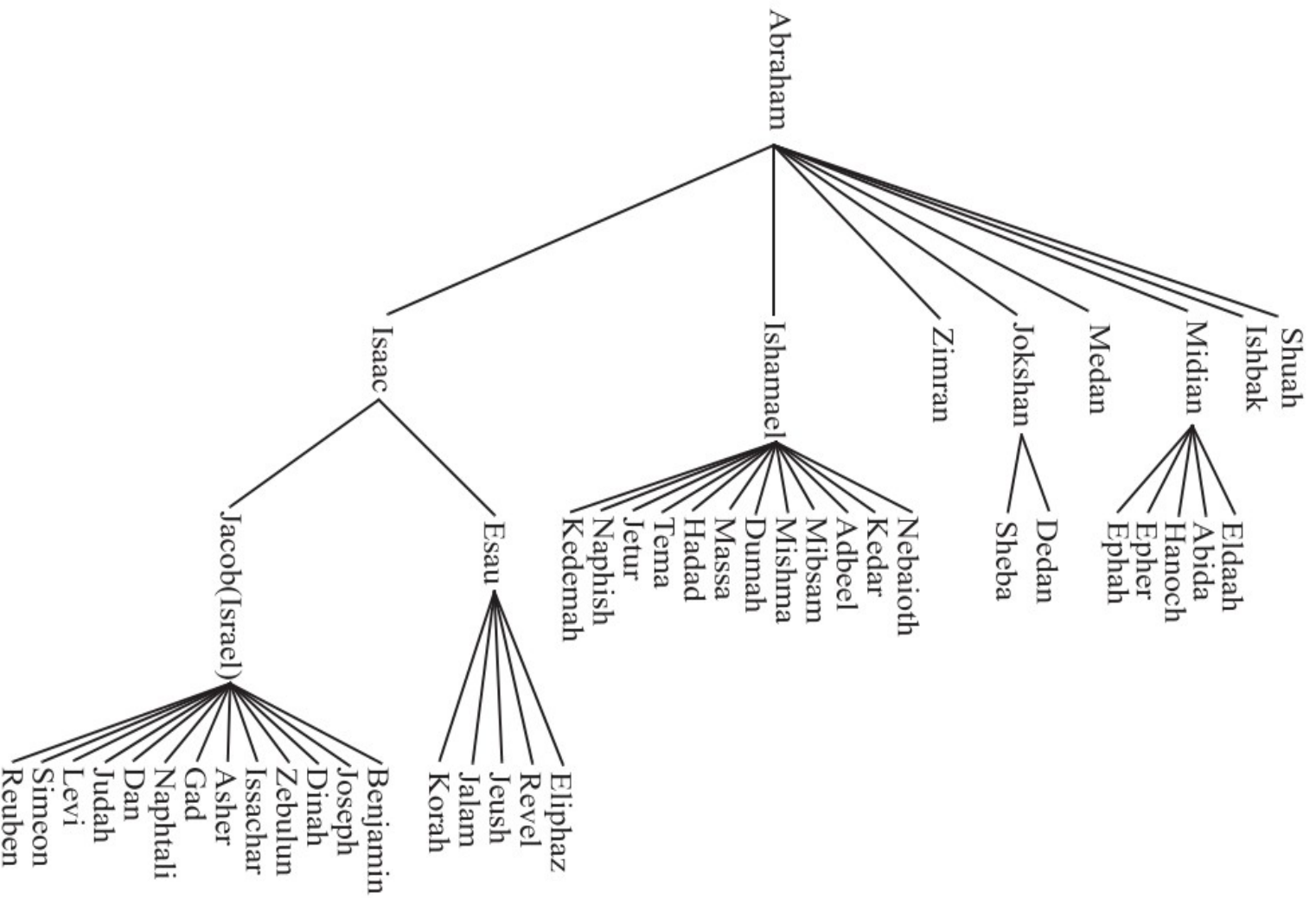
Tree Introduction(next)

Scenario 3: The relationships in a tree are hierarchical, with some objects being “above” and some “below” others. Actually, the main terminology for tree data structures comes from family trees, with the terms “parent,” “child,” “ancestor,” and “descendant” being the most common words used to describe relationships. We show an example of a family tree. This is a figure showing some descendants of Abraham, as recorded in Genesis, chapters 25–36



Linear data structures like arrays, linked list, Queue and Stack can not store the data provided in the image.





Tree and Linked List Comparison



A tree has common behaviour with Linked list because in a tree we need to know the root(from the root we can traverse the tree) and in a linked list we need to know the head(from head we can traverse the linked list). In simple/singly linked list the node stores the link to one node, but in the tree the node will store the address of many nodes.



Why Trees



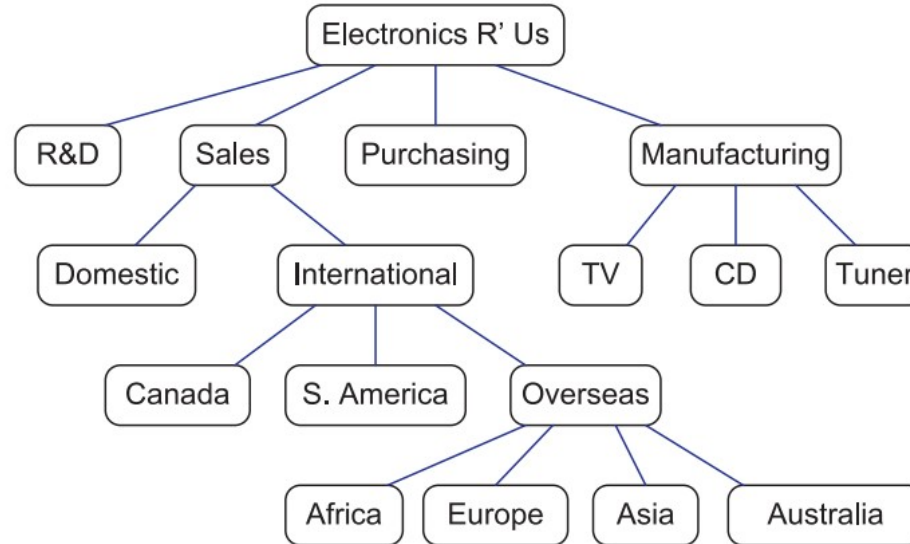
Other data structures such as arrays, linked list, stack, and queue are linear data structures that store data sequentially. In order to perform any operation in a linear data structure, the time complexity increases with the increase in the data size. But, it is not acceptable in today's computational world.

Different tree data structures allow quicker and easier access to the data as it is a non-linear data structure.



Tree definitions

A tree is an abstract data type that stores elements hierarchically and is a connected acyclic collection of nodes and edges. With the exception of the top element, each element in a tree has a parent element and zero or more children elements. A tree is usually visualized by placing elements inside ovals or rectangles, and by drawing the connections between parents and children with straight lines. We typically call the top element the root of the tree, but it is drawn as the highest element, with the other elements being connected below (just the opposite of a botanical tree).



Definition and properties

Formally, we define a tree T as an abstract data type that stores elements hierarchically and consists of a set of nodes storing elements such that the nodes have a parent-child relationship that satisfies the following properties:

1. If T is nonempty, it has a special node, called the root of T , that has no parent.
2. Each node v of T different from the root has a unique parent node w ; every node with parent w is a child of w .
3. Two nodes that are children of the same parent are siblings. A node v is external if v has no children. A node v is internal if it has one or more children. External nodes are also known as leaves.



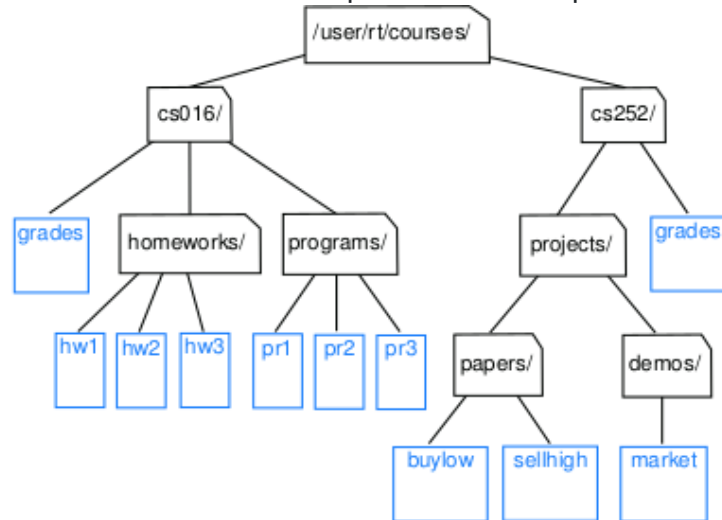
Edges and paths

A node u is an ancestor of a node v if $u \rightarrow v$ or u is an ancestor of the parent of v . Conversely, we say that a node v is a descendant of a node u if u is an ancestor of v .

1. `cs252/` is an ancestor of `papers/`, and `pr3` is a descendant of `cs016/`. The subtree of T rooted at a node v is the tree consisting of all the descendants of v in T (including v itself).
2. the subtree rooted at `cs016/` consists of the nodes `cs016/`, `grades`, `homeworks/`, `programs/`, `hw1`, `hw2`, `hw3`, `pr1`, `pr2`, and `pr3`.

An edge of tree T is a pair of nodes (u, v) such that u is the parent of v , or vice versa. A path of T is a sequence of nodes such that any two consecutive nodes in the sequence form an edge.

3. The path `(cs252/, projects/, demos/, market)`



Tree terminologies

1. Node

A node is an entity that contains a key or value and pointers to its child nodes. The last nodes of each path are called **leaf nodes** or **external nodes** that do not contain a link/pointer to child nodes. The node having at least a child node is called an **internal node**.

2. Edge

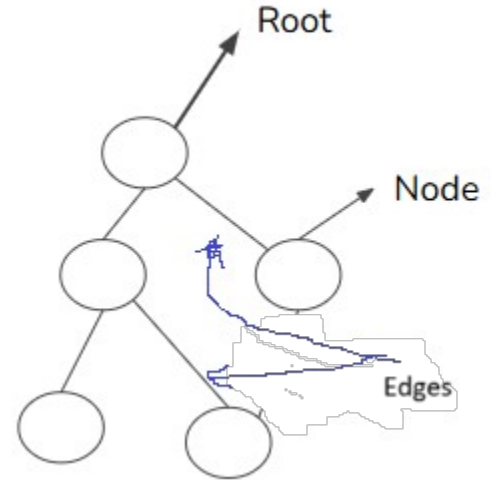
It is a link between two nodes

3. Root

It is the topmost node of a tree.

4. Height of node

The height of a node is the number of edges from the node to the deepest leaf (ie. the longest path from the node to a leaf node).



Tree terminologies

5. Depth of a Node

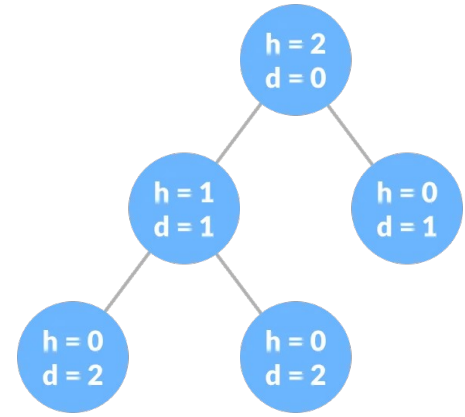
The depth of a node is the number of edges from the root to the node.

6. Height of a Tree

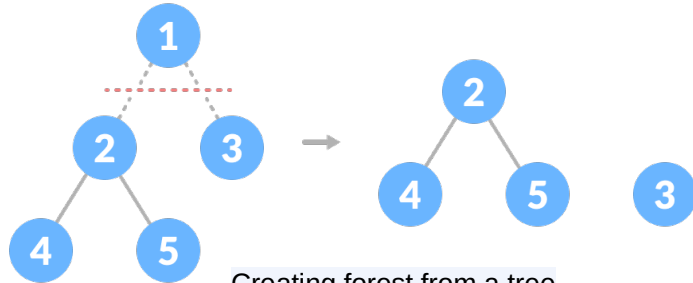
The height of a Tree is the height of the root node or the depth of the deepest node.

7. Forest

A collection of disjoint trees is called a forest.



Depth and Height of a node



Creating forest from a tree



Tree terminologies(next)

8. Descendant

Descendant means children, grand children and great grandchildren and the process continue like that.

9. Ancestors: Ancestor is a parent, grandparent and grand grand parent and so on.

10. Parent : The node which has a branch from it to any other node is called a parent

11. Sibling: nodes which belong to same Parent are called as **SIBLINGS**. In simple words, the nodes with the same parent are called Sibling nodes.

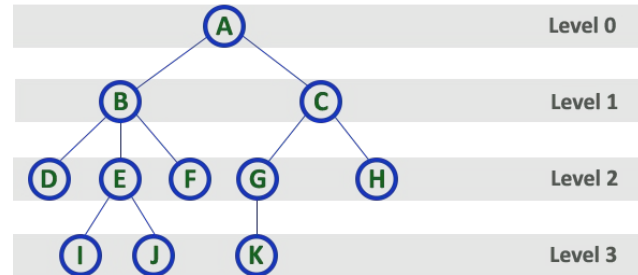
12. Leaf

In a tree data structure, the node which does not have a child is called as **LEAF Node**. In simple words, a leaf is a node with no child.

13. Level

In a tree data structure, the root node is said to be at Level 0 and the children of root node are at Level 1 and the children of the nodes which are at Level 1 will be at Level 2 and so on...

In simple words, in a tree each step from top to bottom is called as a Level and the Level count starts with '0' and incremented by one at each level (Step).



Creating a tree

```
1. #include<iostream>
2. #include<vector>
3. #include<queue>
4. using namespace std;

5. template<typename T>
6. class TreeNode{
7.     public:
8.         T data;
9.         vector<TreeNode<T>*> children;
10.        TreeNode(T data){
11.            this->data = data;
12.        }

13.};
```

```
TreeNode<int>* takeInputLevelWise(){
    int rootData;
    cout<<"Enter root data"<<endl;
    cin>>rootData;
    TreeNode<int>* root = new TreeNode<int>(rootData);
    queue<TreeNode<int>*> q;
    q.push(root);

    while(!q.empty()){
        TreeNode<int>* f = q.front();
        q.pop();

        cout<<"Enter no of children of "<<f->data<<endl;
        int n;
        cin>>n;
        for(int i=1;i<=n;i++){
            int childData;
            cout<<"Enter "<<i<<" th child of "<<f->data<<endl;
            cin>>childData;

            TreeNode<int>* child = new TreeNode<int>(childData);
            q.push(child);
            f->children.push_back(child);
        }
    }
    return root;
}
```

Taking input



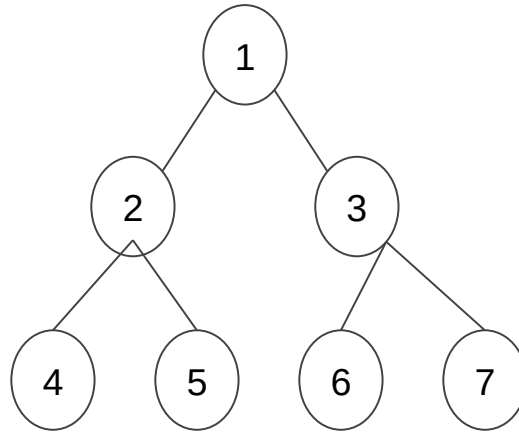
Print Tree Recursively

```
void printTree(TreeNode<int>* root){  
    if(root==NULL){  
        return ;  
    }  
    cout<<root->data<<": ";  
    for(int i=0;i<root->children.size();i++){  
        cout<<root->children[i]->data<<" ,";  
    }  
    cout<<endl;  
  
    for(int i=0;i<root->children.size();i++){  
        printTree(root->children[i]);  
    }  
}
```



Tree Traversing Algorithms

1. Level Order Traversal
2. Depth Level Traversal
 - a. Inorder Traversal
 - b. PreOrder Traversal
 - c. PostOrder Traversal



InOrder Traversal(LDR)

4	2	5	1	6	3	7
---	---	---	---	---	---	---

PreOrder Traversal(DLR)

1	2	4	5	3	6	7
---	---	---	---	---	---	---

PostOrder Traversal(LRD)

4	5	2	6	7	3	1
---	---	---	---	---	---	---



Level Order Traversal: Printing a tree using levels

```
void printTreeLevelWise(TreeNode<int>* root){
    queue<TreeNode<int>*> q;
    q.push(root);
    int counter=0;
    while(!q.empty()){
        TreeNode<int>* f = q.front();
        counter++;
        q.pop();
        cout<<f->data<<": ";
        for(int i=0;i<f->children.size();i++){
            cout<<f->children[i]->data<<" ";
        }
        cout<<endl;

        for(int i=0;i<f->children.size();i++){
            q.push(f->children[i]);
        }
    }
    //Print the value of the counter when the Queue is empty
    cout<<"The number of nodes is "<<counter<<endl;
}
```

In Level Order Traversal, a node is printed and then its children

```
int main(){
    TreeNode<int>* root =
    takeInputLevelWise();
    cout<<"Print tree
    recursively"<<endl;
    printTree(root);
    cout<<"Printing Level wise"<<endl;
    printTreeLevelWise(root);
    return 0;
}
```



Counting nodes of a tree with or without

```
int countNodes(TreeNode<int>* root){
    if(root==NULL){ /// edge case
        return 0;
    }
    int ans = 1;
    for(int i=0;i<root->children.size();i++){
        ans += countNodes(root->children[i]);
    }
    return ans;
}
```

```
int countNodes(TreeNode<int>* root){
    if(root==NULL){ /// edge case
        return 0;
    }
    queue<TreeNode<int>*> q;
    q.push(root);
    int counter=0;
    while(!q.empty()){
        TreeNode<int>* f = q.front();
        counter++;
        q.pop();
        for(int i=0;i<f->children.size();i++){
            q.push(f->children[i]);
        }
    }
    return counter;
}
```

Height of a tree

```
int height(TreeNode<int>* root){
    if(root==NULL){ /// edge case
        return 0;
    }
    int mx = 0; /// maximum height
    for(int i=0;i<root->children.size();i++){
        mx = max(mx, height(root->children[i]));
    }
    return mx + 1;
}
```



Counting Leaf Nodes with and without recursion

A leaf node is a node without children

```
int countLeafNodes(TreeNode* root){
    if(root==NULL){ //edge case
        return 0;
    }
    if(root->children.size()==0){
        return 1;
    }
    int ans = 0;
    for(int i=0;i<root->children.size();i++){
        ans += countLeafNodes(root->children[i]);
    }
    return ans;
}
```

```
int countLeaves(TreeNode<int>* root){
    queue<TreeNode<int>*> q;
    q.push(root);
    int leaves=0;
    while(!q.empty()){
        TreeNode<int>* f = q.front();

        if(f->children.size()==0){
            leaves++;
        }
        q.pop();
        for(int i=0;i<f->children.size();i++){
            q.push(f->children[i]);
        }
    }
    return leaves;
}
```



Depth Level Traversal

InOrder Traversal

```
void inorder(TreeNode<int> *root)
{
    if (root == NULL)
        return;

    // Total children count
    int total=root->children.size();

    // All the children except the last
    for (int i = 0; i < total - 1; i++)
        inorder(root->children[i]);

    // Print the current node's data
    cout<< root->data << " ";

    // Last child
    inorder(root->children[total - 1]);
}
```

1. First, visit all the nodes in the left subtree
2. Then the root node
3. Visit all the nodes in the right subtree



PreOrder Traversal

```
void preOrder(TreeNode<int>* root){  
    if(root==NULL){ /// edge case  
        return ;  
    }  
    cout<<root->data<<" ";  
    for(int i=0;i<root->children.size();i++){  
        preOrder(root->children[i]);  
    }  
}
```

Preorder traversal

1. Visit root node
2. Visit all the nodes in the left subtree
3. Visit all the nodes in the right subtree



PostOrder Traversal

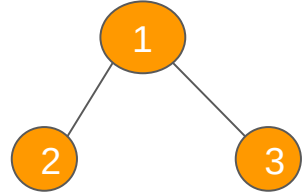
```
void postOrder(TreeNode<int>* root){  
    if(root==NULL){ /// edge case  
        return ;  
    }  
    for(int i=0;i<root->children.size();i++){  
        postOrder(root->children[i]);  
    }  
    cout<<root->data<<" ";  
}
```

PostOrder traversal

1. Visit all the nodes in the left subtree
2. Visit all the nodes in the right subtree
3. Visit the root node



Delete the Tree to free the memory



```
void deleteTree(TreeNode<int>* root){  
    if(root==NULL){ /// edge case  
        return ;  
    }  
    for(int i=0;i<root->children.size();i++){  
        deleteTree(root->children[i]);  
    }  
    delete root;  
}
```

PostOrder delete

1. Delete all children
2. Delete root

Delete Constructor

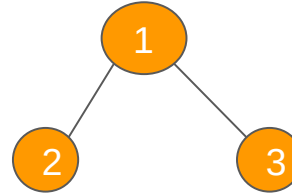
```
#include<iostream>
#include<vector>

#include<queue>

using namespace std;

template<typename T>
class TreeNode{
public:
    T data;
    vector<TreeNode<T>*> children;
    TreeNode(T data){
        this->data = data;
    }
    ~TreeNode(){
        for(int i=0;i<children.size();i++){
            delete children[i];
        }
    }
};
```

To free memory using the destructor, we need to first delete its children. The destructor will first delete the root nodes which are leaves and continue backward.



Delete 1

Will call delete 2

Since 2 is a leaf it will be deleted

After delete 2, the destructor will

delete 3 and

Lastly delete 1



Types of Tree

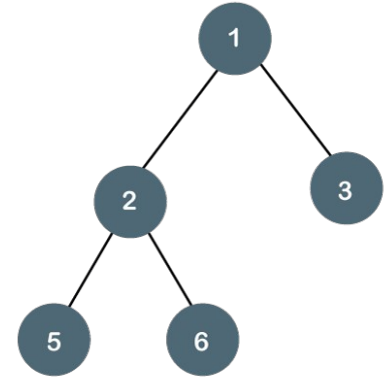
In the following section we are going to discuss 4 types of Tree:

1. Binary tree
2. Binary Search Tree
3. Ternary tree
4. AVL Tree
5. B-Tree



Binary Tree

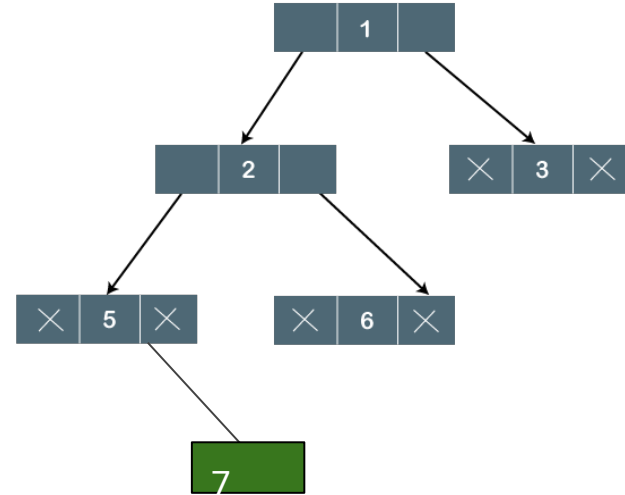
The binary tree is a type of tree data structure where every parent node has a maximum of two child nodes. As the name suggests, binary means two, therefore, each node can have 0, 1, or 2 nodes. A binary tree data structure is shown in right Figure . Node 1 in the tree contains two pointers, one for each child node. Node 2 again has two pointers each for the two child nodes. Whereas node 3, 5, and 6 are leaf nodes and therefore, have null pointers on both left and right parts.



This type of tree data structure allows a maximum of two children for each parent. The children are known as the right kid and the left kid. A binary tree is more popular than other trees in data structure. Applying several limitations and characteristics in a Binary tree, other tree structures like BST (Binary Search Tree), AVL tree, RBT tree, etc. are also used

Binary Tree

In the following tree on the left, node 1 contains two pointers, i.e., left and a right pointer pointing to the left and right node respectively. The node 2 contains both the nodes (left and right node); therefore, it has two pointers (left and right). The nodes 3, 5 and 6 are the leaf nodes, so all these nodes contain **NULL** pointer on both left and right parts.



Properties of Binary Tree

- At each level of i , the maximum number of nodes is 2^i .
 - The height of the tree is defined as the longest path from the root node to the leaf node. The tree which is shown above has a height equal to 3. Therefore, the maximum number of nodes at height 3 is equal to $(1+2+4+8) = 15$. In general, the maximum number of nodes possible at height h is $(2^0 + 2^1 + 2^2 + \dots + 2^h) = 2^{h+1} - 1$.
- The minimum number of nodes possible at height h is equal to **$h+1$** .



Example1: Binary Tree

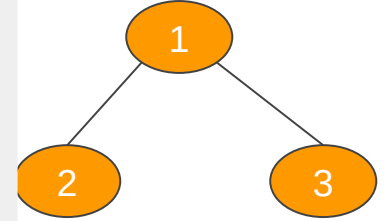
```
#include<iostream>
#include<vector>

#include<queue>

using namespace std;
template<typename T>
class BTreeNode{
public:
    T data;
    BTreeNode* left;
    BTreeNode* right;
    BTreeNode(T data){
        this->data = data;
        left = NULL;
        right = NULL;
    }
    ~BTreeNode(){
        delete left;
        delete right;
```

To create the tree on the left, we need the following:

1. Create a class Binary Tree Node(BTNode) with:
Datatype data
Two Nodes Link(left, right).
2. We need to initialise using a constructor:
data with provided data,
left with NULL,
right with NULL,



Sample Binary Tree

```
void printTree(BTNode<int>* root){
    if(root==NULL){ /// base case
        return;
    }
    cout<<root->data<<": ";
    if(root->left!=NULL){
        cout<<"L"<<root->left->data;
    }
    if(root->right!=NULL){
        cout<<"R"<<root->right->data;
    }
    cout<<endl;

    printTree(root->left);
    printTree(root->right);
}
```

```
int main(){
    BTreeNode<int>* root = new BTreeNode<int>(1);
    BTreeNode<int>* n1 = new BTreeNode<int>(2);
    BTreeNode<int>* n2 = new BTreeNode<int>(3);
    root->left = n1;
    root->right = n2;
    printTree(root);
    delete root;
    return 0;
}
```

Output

1: L2 R3
2:
3:



Take Input Level Wise for Binary Tree

```
#include<iostream>
#include<vector>

#include<queue>

using namespace std;
template<typename T>
class BTreeNode{
public:
    T data;
    BTreeNode* left;
    BTreeNode* right;
    BTreeNode(T data){
        this->data = data;
        left = NULL;
        right = NULL;
    }
    ~BTreeNode(){
        delete left;
        delete right;
```

```
BTreeNode<int>* takeInputLevelWise(){
    int rootData;
    cout<<"Enter root data"<<endl;
    cin>>rootData;
    BTreeNode<int>* root = new BTreeNode<int>(rootData);
    queue<BTreeNode<int>*> q;
    q.push(root);
    while(!q.empty()){
        BTreeNode<int>* f = q.front();
        q.pop();
        cout<<"Enter left child of "<<f->data<<endl;
        int leftChildData;
        cin>>leftChildData;
        if(leftChildData != -1){//Enter -1 if no left
            BTreeNode<int>* child = new BTreeNode<int>(leftChildData);
            q.push(child);
            f->left = child;
        }
        cout<<"Enter right child of "<<f->data<<endl;
        int rightChildData;
        cin>>rightChildData;
        if(rightChildData != -1){//Enter -1 if no right
            BTreeNode<int>* child = new BTreeNode<int>(rightChildData);
            q.push(child);
            f->right = child;
        }
    }
    return root;
}
```

//1.Create

//2.Push

//3. Connect with Parent

```
void printTree(BTreeNode<int>* root){
    if(root==NULL){ // base case
        return;
    }
    cout<<root->data<<" ";
    if(root->left !=NULL){
        cout<<"L"<<root->left->data;
    }
    if(root->right!=NULL){
        cout<<"R"<<root->right->data;
    }
    cout<<endl;
    printTree(root->left);
    printTree(root->right);
}
```

```
int main(){
    BTreeNode<int>* root = new BTreeNode<int>(1);
    BTreeNode<int>* n1 = new BTreeNode<int>(2);
    BTreeNode<int>* n2 = new BTreeNode<int>(3);
    root->left = n1;
    root->right = n2;
    printTree(root);
    BTreeNode<int>* root2=takeInputLevelWise();
    printTree(root2);
    delete root;
    return 0;
```



Example 2: Simple Binary Tree Program

```
#include <iostream>
using namespace std;
class Node{
public:
    int data;
    Node* left;
    Node* right;
public:
    Node(int d){
        this->data=d;
        this->left=NULL;
        this->right=NULL;
    }
};
```

```
class BinaryTree{
private:
    Node * root;
public:
    BinaryTree(){
        this->root=NULL;
    }
    Node * addNode(Node * subTree,int data,bool isRight){
        Node * newNode=new Node(data);
        if(subTree==NULL){
            root=newNode;
            return root;
        }
        if(isRight==true){
            subTree->right=newNode;
        }
        else
            subTree->left=newNode;
        return newNode;
    }
    void display(Node * root){
        if(root==NULL){
            return;
        }
        cout<<root->data<<" ";
        print(root->left);
        print(root->right);
    }
};
```

```
int main()
{
    BinaryTree bt;
    Node
    *root=bt.addNode(NULL,1,false);
    Node *left=bt.addNode(root,2,false);

    Node
    *rightt=bt.addNode(root,3,true);
    Node
    *leftLeft=bt.addNode(left,4,false);
    Node
    *leftRihgt=bt.addNode(left,5,true);
    Node
    *righttLeft=bt.addNode(rightt,6,false);
    Node
    *righttRight=bt.addNode(rightt,7,true);
    bt.display(root);
    return 0;
}
```



Binary Tree Traversal

Depth First Traversals:

Inorder Traversal

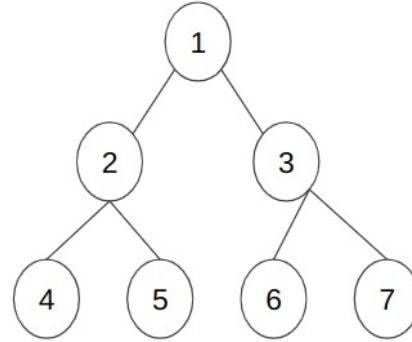
1. First, visit all the nodes in the left subtree
2. Then the root node
3. Visit all the nodes in the right subtree

Preorder traversal

4. Visit root node
5. Visit all the nodes in the left subtree
6. Visit all the nodes in the right subtree

PostOrder traversal

7. Visit all the nodes in the left subtree
8. Visit all the nodes in the right subtree
9. Visit the root node



InOrder Traversal						
4	2	5	1	6	3	7
PreOrder Traversal						
1	2	4	5	3	6	7
PostOrder Traversal						
4	5	2	6	7	3	1

Visualizer

2 . <https://visualgo.net/en/bst>



Example3: Program for Binary Tree Traversal

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    struct Node *left, *right;
    Node(int data) {
        this->data = data;
        left = right = NULL;
    }
};

// Inorder traversal
void inorderTraversal(Node* node) {
    if (node == NULL)
        return;
    inorderTraversal(node->left);
    cout << node->data << "->";
    inorderTraversal(node->right);
}
```

```
// Preorder traversal
void preorderTraversal(Node* node) {
    if (node == NULL)
        return;

    cout << node->data << "->";
    preorderTraversal(node->left);
    preorderTraversal(node->right);
}

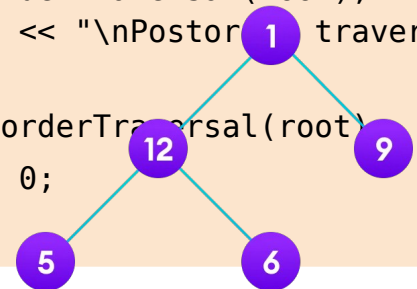
// Postorder traversal
void postorderTraversal(Node* node) {
    if (node == NULL)
        return;

    postorderTraversal(node->left);
    postorderTraversal(node->right);
    cout << node->data << "->";
}
```

```
int main() {
    struct Node* root = new
    Node(1);
    root->left = new Node(12);
    root->right = new Node(9);
    root->left->left = new
    Node(5);
    root->left->right = new
    Node(6);

    cout << "Inorder traversal ";
    inorderTraversal(root);
    cout << "\nPreorder traversal
    ";
    preorderTraversal(root);
    cout << "\nPostorder traversal
    ";
    postorderTraversal(root);
    return 0;
```

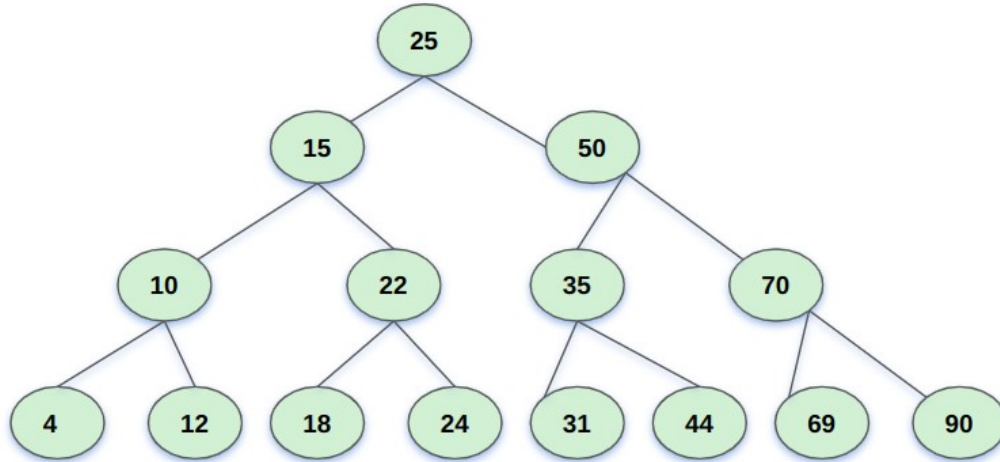
Inorder traversal 5->12->6->1->9->
Preorder traversal 1->12->5->6->9->
Postorder traversal 5->6->12->9->1->



Output:



Example 2: Binary Tree Traversal



Inorder(LDR)
4,10,12,15,18,22,24,25,31,35,44,50,69,70,90
Preorder(DLR)
25,15,10,4,12,22,18,24,50,35,31,44,70,69,90
Postorder(LRD)
4,12,10,18,24,22,15,31,44,35,69,90,70,50,25



Types of Binary trees: 1. Binary Search Tree

These tree structures are non-linear and one node is connected to a number of nodes. The node can be connected to at most two child nodes. It is called a binary search tree because:

The following is the definition of Binary Search Tree(BST):

Binary Search Tree is a node-based binary tree data structure which has the following properties:

1. The left subtree of a node contains only nodes with keys less than the node's key.
 2. The right subtree of a node contains only nodes with keys greater than the node's key.
 3. The left and right subtree each must also be a binary search tree.
 4. There must be no duplicate nodes.
 5. Each node has a maximum of two child nodes.
 6. It can be used to search an element in $O(\log(n))$ time and hence known as a search tree
- The above properties of Binary Search Tree provides an ordering among keys so that the operations like search, minimum and maximum can be done fast. If there is no ordering, then we may have to compare every key to search for a given key.



Searching a key

For searching a value, if we had a sorted array we could have performed a binary search. Let's say we want to search a number in the array, in binary search, we first define the complete list as our search space, the number can exist only within the search space.

1. Now we compare the number to be searched or the element to be searched with the middle element (median) of the search space
2. if the record being searched is less than the middle element, we go searching in the left half,
3. else we go searching in the right half,
4. in case of equality we have found the element.

In binary search we start with 'n' elements in search space and if the mid element is not the element that we are looking for, we reduce the search space to 'n/2' we keep reducing the search space until we either find the record that we are looking for or we get to only one element in search space and be done with this whole reduction.



Searching a key (next...)

Search operations in binary search tree will be very similar. Let's say we want to search for the number, we start at the root, and then we compare the value to be searched with the value of the root, if it's equal we are done with the search if it's smaller we know that we need to go to the left subtree because in a binary search tree all the elements in the left subtree are smaller and all the elements in the right subtree are larger.

Searching an element in the binary search tree is basically this traversal, at each step we go either left or right and at each step we discard one of the sub-trees. If the tree is balanced (we call a tree balanced if for all nodes the difference between the heights of left and right subtrees is not greater than one) we start with a search space of ' n ' nodes and as we discard one of the sub-trees, we discard ' $n/2$ ' nodes so our search space gets reduced to ' $n/2$ '. In the next step we reduce the search space to ' $n/4$ ' and we repeat until we find the element or our search space is reduced to only one node. The search here is also a binary search hence the name; Binary Search Tree.



Search a Node In BST

```
Node* search(struct node* root, int key)
{
    // Base Cases: root is null or key is present at root
    if (root == NULL || root->key == key)
        return root;

    // Key is greater than root's key
    if (root->key < key)
        return search(root->right, key);

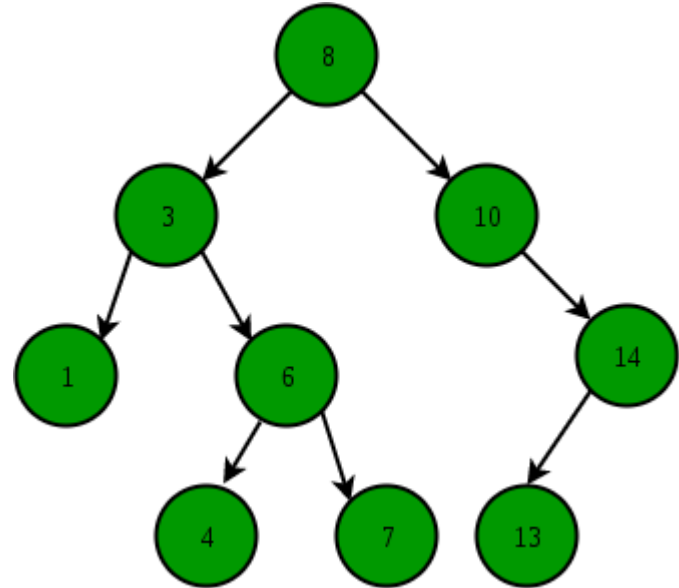
    // Key is smaller than root's key
    return search(root->left, key);
}
```

```
Node* search(struct node* root, int
key)
{
    while(root!=NULL){
        if(root->data == key) return root;
        else if(key > root->data){
            root = root->right;
        }else{
            root = root->left;
        }
    }
    return NULL;
}
```



Illustration to search 6 in below tree:

1. Start from the root.
2. Compare the searching element with root, if less than root, then recursively call left subtree, else recursively call right subtree.
3. If the element to search is found anywhere, return true, else return false.

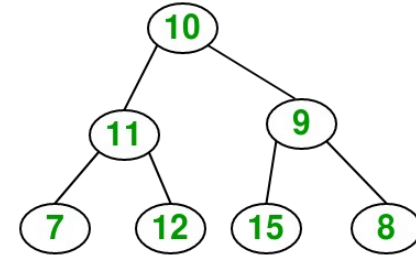
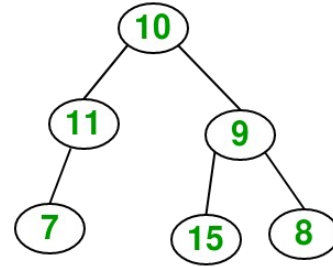


Insertion in a Binary Tree in level order

Given a binary tree and a key, insert the key into the binary tree at the first position available in level order.

The idea is to do iterative level order traversal of the given tree using a queue. If we find a node whose left child is empty, we make a new key as the left child of the node.

Else if we find a node whose right child is empty, we make the new key as the right child. We keep traversing the tree until we find a node whose left or right child is empty.



After inserting 12

Binary Search Program

// C++ program to insert element in Binary Tree

```
#include <iostream>
#include <queue>
using namespace std;
```

/* A binary tree node has data, pointer to left child and a pointer to right child */

```
struct Node {
    int data;
    Node* left;
    Node* right;
};
```

/* Function to create a new node */

```
Node* createNode(int data)
{
    Node* newNode = new Node();
    if (!newNode) {
        cout << "Memory error\n";
        return NULL;
    }
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}
```

```
Node* insert(Node* node, int key)
{
    // If the tree is empty, return a new node
    if (node == NULL)
        return createNode(key);

    // Otherwise, recur down the tree
    if (key < node->data)
        node->left = insert(node->left, key);
    else if (key > node->data)
        node->right = insert(node->right, key);
    // Return the node pointer
    return node;
}
```

```
Node* search(Node* root, int key)
{
    // Base Cases: root is null or key is present at root
    if (root == NULL || root->data == key)
        return root;
    // Key is greater than root's key
    if (root->data < key)
        return search(root->right, key);
    // Key is smaller than root's key
    return search(root->left, key);
}
```

```
/* Inorder traversal of a binary tree */
void inorder(Node* root)
{
    if (root == NULL)
        return;
    inorder(root->left);
    cout << root->data << ' ';
    inorder(root->right);
}
```

// Driver code

```
int main()
{
    Node* root = createNode(10);
    root->left = createNode(11);
    root->left->left = createNode(7);
    root->right = createNode(9);
    root->right->left = createNode(15);
    root->right->right = createNode(8);
    cout << "Inorder traversal before insertion: ";
    inorder(root);
    cout << endl;
    int key = 12;
    root = insert(root, key);
    cout << "Inorder traversal after insertion: ";
    inorder(root);
    cout << endl;
    return 0;
}
```

Output

Inorder traversal before insertion: 7 11 10 15 9 8
Inorder traversal after insertion: 7 11 12 10 15 9 8



Minimum vs Maximum Node in BST

Minimum in BST

For the minimum, we need to return the leftmost element.

```
int minNode( Node *root)
{
    if(root==NULL)
        return -1;
    if( root->left!=NULL)
    {
        root=root->left;
    }
    return root->data;
}
```

The Time Complexity is $O(n)$

Maximum in BST

For the maximum, we need to return the right most element

```
int maxNode( Node *root)
{
    if(root==NULL)
        return -1;
    if( root->right!=NULL)
    {
        root=root->right;
    }
    return root->data;
}
```

The Time Complexity is $O(n)$

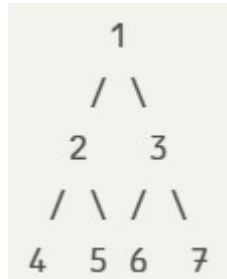


Types of Binary trees: 2. **Full Binary Tree**

Definition: Every node has either 0 or 2 children (no node has only one child)

Also called: Proper or Strict Binary Tree.

Example:



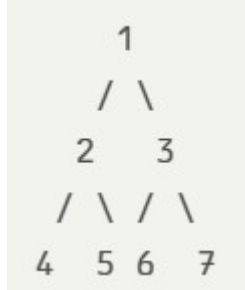
Use Cases: Heaps, some expression trees

Types of Binary trees: 3. **Perfect Binary Tree**

Definition: All internal nodes have exactly two children, and all leaf nodes are at the same level.

Every level is completely filled.

Example:



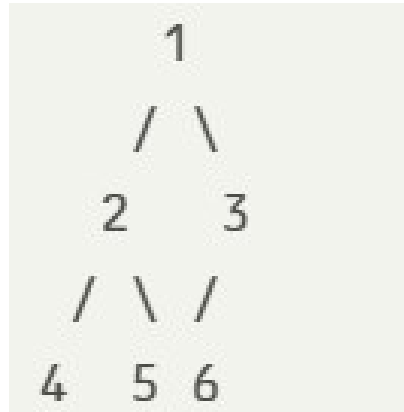
Use Cases: Efficient storage and traversal.

Types of Binary trees: 4. **Complete Binary Tree**

Definition: All levels are completely filled except possibly the last, which is filled from left to right.

Nodes are as far left as possible.

Example:



Types of Binary trees: 5. **Heap**

A heap is a specific type of binary tree that follows two main properties: **the complete binary tree property** and **the heap property**. A complete binary tree had been explained in the slide above. The heap property dictates that the value of each node is either greater than or equal to (max-heap) or less than or equal to (min-heap) the values of its children.

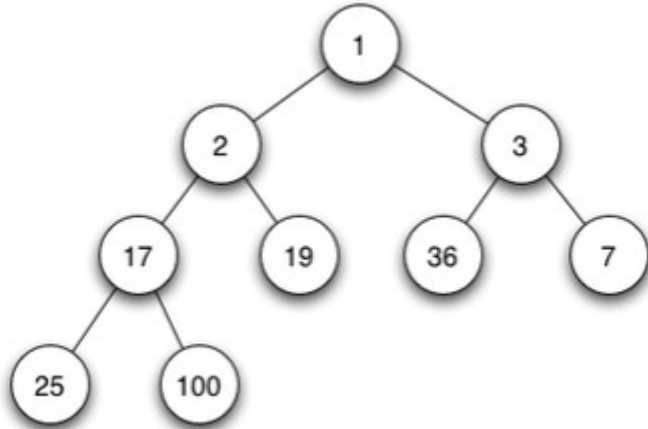
Use cases: Priority queues, heapsort.



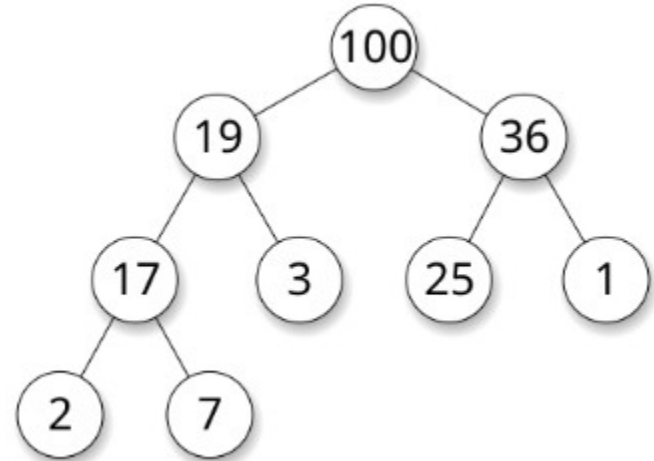
Heap

Heap Examples:

Min-Heap:



Max-Heap:

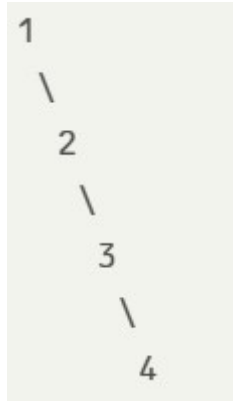


Types of Binary trees: 6. **Degenerate (Pathological) Binary Tree**

Definition: Each parent node has only one child.

Structure: Resembles a linked list.

Example:



Types of Binary trees: 7. **Skewed Binary Tree**

Definition: Special case of degenerate tree; all nodes lean to one side.

Types:

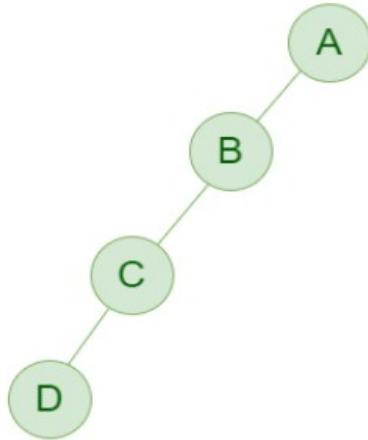
- **Left-skewed:** All nodes have only left child.
- **Right-skewed:** All nodes have only right child.



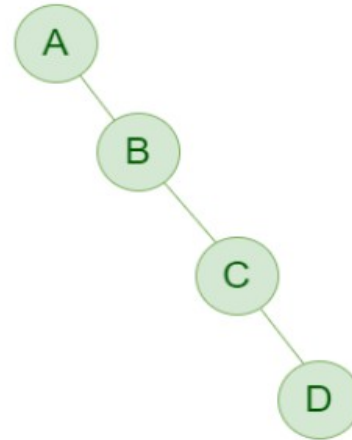
Skewed Binary Tree

Examples:

Left-Skewed Binary Tree



Right-Skewed Binary Tree



Types of Binary trees: 8. **Balanced Binary Tree**

Definition: For every node, the height difference between left and right subtrees is at most one

Use cases: Databases, search engines



Example of balanced and unbalanced trees:

